



Bug Bounty Report

**CarthaVault: ERC-20 Shares Become Transferable
After Lock Expiry but No Exit Path Exists for the
Recipient**

alva-p.xyz

August 2026

Prepared by: alva-p

Lead Researcher: Pineda Alvaro

Summary

`CarthaVault` allows ERC-20 share transfers after a position's lock expires, but every permissionless exit remains tied to the original depositor's position. After the original holder transfers the shares, their payout is capped to zero; the recipient has no position and therefore cannot redeem. Unless the recipient cooperates by returning the shares, neither party has a permissionless recovery path.

Severity: Medium

Affected code: `cartha-vaults/src/CarthaVault.sol`

Root Cause

`_update()` blocks a transfer only while the original position remains locked by time:

```
1 bool locked = block.timestamp < position.start + uint256(position.  
    lockDays) * 1 days;  
2 if (locked) revert PositionLocked();
```

After expiry, the shares behave as transferable ERC-20 tokens even though redemption remains bound to the position. The original owner's exits cap shares to `min(position.shares, balanceOf(owner))`; after a full transfer this becomes zero.

`release()` then reverts:

```
1 if (payout == 0) revert InsufficientVaultBalance();
```

The queued path silently skips the zero payout without clearing state:

```
1 if (payout == 0 || payout > vaultBalance) continue;
```

Meanwhile, the recipient cannot use `release()`, `requestRelease()`, queued withdrawals, or `forceExit()` because every path requires `position.exists` for the caller. A recipient who never deposited has no position.

Happy Path vs. Attack Path

Happy Path

1. Alice deposits and locks 100,000 USDC, receiving `S` shares tied to her position.
2. After the lock expires, Alice keeps the shares in her account.
3. Alice calls `release()`; her position shares and token balance match.
4. The vault burns `S` and returns the proportional assets.

Failure Path

1. Alice deposits and locks 100,000 USDC for 30 days, receiving `S` shares.
2. After 30 days, `_update()` allows Alice to transfer all `S` shares to Bob.
3. Alice calls `release()`. Her balance is zero, so the calculated payout is zero and the call reverts.
4. Alice requests a queued withdrawal. The keeper calculates a zero payout and `continues` without clearing `pendingWithdrawal`.
5. Later requests revert with `PendingWithdrawalExists`.
6. Bob attempts to exit, but every exit path reverts with `PositionNotFound` because Bob never created the position.
7. Recovery requires Bob to return the shares. If Bob is unavailable, a dead address, or a contract unable to cooperate, the funds have no permissionless recovery path.

Impact

The position owner loses permissionless access to the underlying assets after transferring the shares. The recipient holds ERC-20 shares with no redemption function available to them, and the queued withdrawal state can deadlock indefinitely. A force exit with zero shares can delete the original position without paying assets, leaving recovery dependent on privileged intervention.

Proof of Concept

The standalone Foundry reproduction is located at:

```
1 findings/MEDIUM-3-carthaVault-stranded-shares-post-lock-expiry/forge-poc
```

Run:

```
1 forge test --match-contract CarthaVaultStrandedSharesPoC -vv
```

Expected result:

```
1 [PASS] test_ReleaseRevertsAfterShareTransfer()
2 [PASS] test_QueuedWithdrawalDeadlock()
3 [PASS] test_RecipientSharesStranded()
4 [PASS] test_ForceExitZeroPayoutSharesRemainStranded()
5 Suite result: ok. 4 passed; 0 failed
```

Fix Direction

Block outgoing transfers for any account with an active position, including after the time lock expires:

```
1 if (position.exists) {
2 -     bool locked = block.timestamp < position.start + uint256(
      position.lockDays) * 1 days;
3 -     if (locked) revert PositionLocked();
4 +     revert PositionLocked();
5 }
```

Separately, clear `pendingWithdrawal` when queued processing finds a zero payout. That prevents the queue deadlock but does not give recipients a redemption path.

Report Boundary

The transfer must occur after lock expiry, and the original owner can recover if the recipient voluntarily returns the shares. The security failure is that the token exposes a valid transfer while the protocol provides neither party an independent, permissionless exit afterward.